

Reviewer Pack — Fourier Spectral Entropy of Clause-Falsification Polynomial ...

Entry 363fda00c17f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 01:40:46 UTC

Fourier Spectral Entropy of Clause-Falsification Polynomial Bounds Tree-Resolution

Entry ID: 363fda00c17f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 01:40:46 UTC

1. Conjecture

Field A (mathematical branch): Fourier-analytic information theory — Shannon entropy of the normalized Walsh-Hadamard spectrum of a multilinear pseudo-Boolean polynomial (Friedgut-Kalai 1996 FEI conjecture; O'Donnell 2014 'Analysis of Boolean Functions' Ch. 9; Wan-Wang-Yu 2014 on FEI for low-degree functions; Chakraborty-Kulkarni-Lokam-Saurabh 2016 'Upper bounds on Fourier entropy'). The functional $H_F(F) := -\sum_S q_F(S) \cdot \log_2 q_F(S)$, where $q_F(S) := \hat{p}_F(S)^2 / \|p_F\|^2$, is the Shannon entropy of the SPECTRAL probability distribution viewed as a measure on the Boolean character lattice 2^1 . It is distinct from Mansour $L^1 \sum |\hat{p}(S)|$ (linear in $|\hat{p}|$, no log), Bourgain NSM $\sum_S (1 - \rho^{|S|}) \hat{p}(S)^2$ (level-indexed L^2 weight, not entropy), Bonami 4/2 ratio $\|p\|_4 / \|p\|_2$ (norm ratio), Talagrand $\sum_i \sqrt{\inf_i}$ (per-coordinate L^2 -root), term-pair noise log-ratio (Rényi-2-to-4 hypercontractive kernel), and cross-side symdiff mass (truth-table CC-matrix Fourier). It uses log of a normalized ratio of squares, so it admits no polynomial F_q extension (Shannon entropy of a probability distribution is not preserved under polynomial-ring lifts of the Boolean ring), making the bridge Aaronson-Wigderson algebrization-safe. Razborov-Rudich-safe because $H_F(F)$ is a property of the CLAUSE LIST (multiplicities, polarities), so two unsat 3-CNFs F_1, F_2 with identical satisfiability function (both $\equiv \text{FALSE}$) can have very different H_F . An arXiv search 'Fourier entropy' AND ('DPLL' OR 'tree-Resolution' OR 'CNF refutation' OR 'proof complexity') returns 0 direct hits and <5 adjacent papers (all on FEI for ± 1 Boolean f or random-DNF entropy, never on the clause-falsification polynomial of an unsat CNF). **Field B** (complexity object): Tree-like Resolution / DPLL refutation size $t(F)$ for unsatisfiable 3-CNFs F in the Chvátal-Szemerédi 1988 / Ben-Sasson-Wigderson 2001 / Beame-Karp-Pitassi-Saks 2002 / Krajíček 1995 bounded-arithmetic regime, where $\neg F$ is Σ^b_0 and $\log_2 t(F)$ lower-bounds V^0 -witnessing complexity; the canonical Frege/Extended-Frege depth stepping stone via Pudlák-Buss feasible interpolation.

Statement:

Let F be an unsatisfiable 3-CNF on n variables with m clauses. Define $p_F : \{-1, +1\}^n \rightarrow \mathbb{R}$ by $p_F(x) := \sum_{C \in F} \mathbb{1}[x \text{ falsifies } C]$; its Walsh-Hadamard spectrum $\hat{p}_F(S)$ is supported on $|S| \leq 3$ and admits the closed form $\hat{p}_F(\emptyset) = m/8$, $\hat{p}_F(\{i\}) = -\sum_{C \ni i} \varepsilon_i^C / 8$, $\hat{p}_F(\{i, j\}) = \sum_{C \supseteq \{i, j\}} \varepsilon_i^C \varepsilon_j^C / 8$, $\hat{p}_F(\{i, j, k\}) = -\sum_{C = \{i, j, k\}} \varepsilon_i^C \varepsilon_j^C \varepsilon_k^C / 8$ where $\varepsilon_i^C \in \{\pm 1\}$ is the polarity of variable i in clause C . Let $q_F(S) := \hat{p}_F(S)^2 / \|p_F\|^2$ and $H_F(F) := -\sum_{S: |S| \leq 3} q_F(S) \cdot \log_2 q_F(S)$. Conjecture: there exists a universal constant $c > 0$ such that for every unsat 3-CNF F at clause-to-variable ratio $\alpha \in [4.2, 5.0]$,

¹_n

$\log_2 t(F) \geq c \cdot n \cdot H_F(F) / \log_2(1+n+n^2+n^3)$. One counterexample (an unsat 3-CNF whose ratio $\log_2 t(F) \cdot \log_2(1+n+n^2+n^3) / (n \cdot H_F(F))$ is arbitrarily small) refutes it.

Rationale (proposer’s reasoning):

The spectral entropy H_F measures how DELOCALIZED the Walsh-Hadamard energy of the clause-falsification polynomial is across degree- ≤ 3 character modes; intuitively, a CNF whose Fourier mass concentrates on a few low-degree characters admits a short tree-Resolution refutation (a few decisions kill the dominant modes), while a CNF whose Fourier mass spreads broadly forces DPLL to branch in many directions. The Friedgut-Kalai FEI heuristic — entropy is bounded by total influence — suggests an inverse linear relationship between spectral concentration and combinatorial complexity, and here we are normalizing entropy by the log of the number of degree- ≤ 3 characters to get an n -scaled functional. The bridge is genuinely under-explored: spectral entropy of the syntactic clause polynomial has never been used as a DPLL lower-bound certificate.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): fa729b956355e2cd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n \in \{10, 12, 14, 16, 18, 20\}$ with 30 unsat 3-CNF seeds at $\alpha=4.3$, compute $R(F) = \log_2 t(F) \cdot \log_2(1+n+n^2+n^3) / (n \cdot H_F(F))$.
Conjecture SUPPORTED iff $\min_{seed} R \geq 0.05$ at every n AND $\text{Pearson}(\log_2 t, n \cdot H_F / \log_2(1+n+n^2+n^3)) \geq 0.5$.
FALSIFIED if any seed yields $R < 0.01$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.83	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Fourier entropy AND (tree resolution OR DPLL OR CNF refutation) - Walsh Hadamard spectrum unsatisfiable 3-CNF proof complexity - spectral entropy clause falsification polynomial resolution lower bound

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n, m):
        clauses = set()
        variables = list(range(1, n + 1))
        for _ in range(m):
            clause = []
            for _ in range(3):
                var = random.choice(variables)
                polarity = random.choice([-1, 1])
                clause.append((var, polarity))
            clauses.add(tuple(sorted(clause)))
        return clauses

    def is_unsat(cnf):
        # Simple DPLL algorithm to check unsatisfiability
        assignment = [None] * (n + 1)

        def dpll():
            stack = []
            for var in range(1, n + 1):
                if all(abs(clause[var - 1][0]) == abs(var) and clause[var - 1][1] != polarity
                       for clause in cnf):
                    assignment[var] = 1
                    stack.append((var, 1))
                elif all(abs(clause[var - 1][0]) == abs(var) and clause[var - 1][1] != polarity
                        for clause in cnf):
                    assignment[var] = -1
                    stack.append((var, -1))
                else:
                    return False
            while stack:
                var, polarity = stack.pop()
                if not all(abs(clause[var - 1][0]) == abs(var) and clause[var - 1][1] != polarity
                          for clause in cnf):
                    assignment[var] = -polarity
                    stack.append((var, -polarity))
            return True

        return not dpll()

    def walsh_hadamard_transform(cnf):
        n_vars = len(cnf)
        spectrum = [0] * (2 ** n_vars)

        for i in range(1 << n_vars):
            x = [1 if (i >> j) & 1 else -1 for j in range(n_vars)]
            value = sum(1 if all(x[var - 1] == polarity for var, polarity in clause) else -1 for clause in cnf)
            spectrum[i] = value / math.sqrt(len(cnf))

        return spectrum

    def spectral_entropy(spectrum):
        norm_squared = sum(abs(val) ** 2 for val in spectrum)
```

```

entropy = 0
for val in spectrum:
    if val != 0:
        q = abs(val) ** 2 / norm_squared
        entropy -= q * math.log2(q)
return entropy

def tree_resolution(cnf):
    # Simplified tree-resolution algorithm to count leaf nodes
    stack = []
    for var in range(1, n + 1):
        if all(any(abs(clause[var - 1][0]) == abs(var) and clause[var - 1][1] != polarity for clause in cnf)):
            assignment[var] = 1
            stack.append((var, 1))
        elif all(any(abs(clause[var - 1][0]) == abs(var) and clause[var - 1][1] != polarity for clause in cnf)):
            assignment[var] = -1
            stack.append((var, -1))
        else:
            return len(cnf)

    while stack:
        var, polarity = stack.pop()
        if not all(any(abs(clause[var - 1][0]) == abs(var) and clause[var - 1][1] != polarity for clause in cnf)):
            assignment[var] = -polarity
            stack.append((var, -polarity))

    return len(cnf)

n = random.randint(10, 20)
m = math.ceil(4.3 * n)
cnf = generate_3cnf(n, m)

if not is_unsat(cnf):
    return {
        "metric_name": "R(F)",
        "metric_value": float('nan'),
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "unsatisfiable_cnf"
    }

spectrum = walsh_hadamard_transform(cnf)
H_F = spectral_entropy(spectrum)
t_star = tree_resolution(cnf)

if H_F == 0:
    return {
        "metric_name": "R(F)",
        "metric_value": float('nan'),
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "zero_entropy"
    }

R_F = math.log2(t_star) * math.log2(1 + n + n**2 + n**3) / (n * H_F)

```

```

    return {
        "metric_name": "R(F)",
        "metric_value": R_F,
        "instances_tested": 1,
        "conjecture_holds": R_F >= 0.05,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(3, 6)] # Default to first 4 seeds
    results = []

    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {trial}")
        results.append(trial)

    R_F_values = [result["metric_value"] for result in results if not math.isnan(result["metric_value"])]
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(R_F >= 0.05 for R_F in R_F_values) and support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={sum(R_F_values)/len(R_F_values)} std={math.sqrt(sum((x - s)**2 for x, s in zip(R_F_values, support_fraction)))}")
    elif any(result["conjecture_holds"] is False for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"low_R_F\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6e131d60.py", line 139, in <module>
    trial = run_trial(seed)
            ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6e131d60.py", line 111, in run_trial
    spectrum = walsh_hadamard_transform(cnf)
               ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6e131d60.py", line 59, in walsh_hadamard_transform
    spectrum = [0] * (2 ** n_vars)
                ~~~~~^~~~~~
OverflowError: cannot fit 'int' into an index-sized integer

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an OverflowError when attempting to allocate a 2^n -sized array for the Walsh-Hadamard transform, so no $R(F)$ values or Pearson correlations were produced. Without data, neither the support nor the falsification condition can be evaluated. | next: Rewrite walsh_hadamard_transform to exploit the closed-form spectrum (support only on $|S| \leq 3$, $O(n^3)$ coefficients) rather than allocating a 2^n array, then rerun the pre-registered protocol.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	192307
2	preregistration	claude_max	opus	0	0	5535
3	novelty	claude_max	opus	0	0	4190
4	novelty	claude_max	opus	0	0	10667
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11920
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14989
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14825
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16912
9	judge	claude_max	opus	0	0	4190

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 275535 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/363fda00c17f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/363fda00c17f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/363fda00c17f.tar.gz` (if generated)